

ENGENHARIA DE SOFTWARE

SEMANA 1 - AULA 1

Gilberto Falco Netto

Tecnólogo em Análise e Desenvolvimento de Sistemas,
Faculdade Anhanguera - Jundiaí/SP

Sumário

- 1 Visão Geral
- 2 Definição de Engenharia de Software
- 3 Princípios da Engenharia de Software
- 4 Modelo em Cascata
- 5 No Silver Bullet
- 6 Essência e Acidentes
- 7 Métodos Tradicionais
- 8 Limitações dos Métodos Tradicionais
- 9 Propostas de Brooks
- 10 Metodologias Ágeis
- 11 Comparação: Tradicional vs. Ágil
- 12 Vantagens das Metodologias Ágeis

Definição de Engenharia de Software

A literatura nos oferece diversas definições para a Engenharia de Software, desde as sucintas até as mais elaboradas. Porém, a grande maioria utiliza-se de termos como procedimentos, métodos e ferramentas para indicar uma natural convergência: a qualidade do produto a ser desenvolvido.

Definição de Engenharia de Software

Importância

Se o resultado da aplicação de procedimentos, métodos e ferramentas não for uma entrega de qualidade, de pouco terá valido todo o esforço em adotá-los.

Primeiro Passo

Como primeiro passo, convém conhecermos a definição para Engenharia de Software dada por um importante autor dessa área.

Definição de Engenharia de Software

Pressman e Maxim (2016)

A Engenharia de Software abrange um processo, um conjunto de métodos (práticas) e um leque de ferramentas que possibilitam aos profissionais desenvolverem software de altíssima qualidade.

Natureza Dinâmica

Ao analisarmos uma definição, devemos considerar a natureza absolutamente dinâmica da Engenharia de Software.

Evolução

Com o passar dos anos, a importância que um programa de computador assumiu na vida das pessoas e em seus negócios foi sensivelmente alterada e a necessidade por qualidade e agilidade nas entregas cresceu na mesma proporção.

Novos Meios

O surgimento de novos meios de desenvolvimento de software justifica essa evolução. Dominar conceitos da Engenharia de Software é essencial para a prática alinhada com as necessidades da organização.

Adaptação e Definição

Adaptação

Via de regra, aquele que domina um conceito é capaz de adaptá-lo às suas necessidades ou as da organização em que atua, sem o risco de esvaziar sua essência.

Adaptação e Definição

Detalhamento

Uma melhor definição do conceito de Engenharia de Software pede o detalhamento dos termos que o compõem.

Termo: Engenharia

O termo “Engenharia” está relacionado ao projeto e à manufatura de um artefato, ressaltando a importância dos requisitos e das especificações do artefato.

Produção de Programas

Por não possuir uma forma física, um programa de computador não passa por processo de manufatura tradicional. A produção de programas de computador possui situações particulares que requerem soluções especializadas.

Definição de Software

Definição Satisfatória

Uma definição satisfatória para software inclui:

Elementos

- Instruções que, quando executadas, produzem a função desejada.
- Estruturas de dados que possibilitam os programas manipularem a informação.
- Documentação relativa ao sistema.

Princípios da Engenharia de Software

Importância dos Princípios

Formada a base conceitual da disciplina, avançamos agora para elementos que costumamos chamar de princípios da Engenharia de Software, justamente por darem uma feição própria a ela e por suportarem suas práticas. Eles ajudam a estruturar os processos e a padronizar as soluções propostas por esse ramo da Computação.

Publicação

Esses princípios foram propostos pelo SWEBOK (Software Engineering Body of Knowledge), uma importante publicação do IEEE na área.

Organização Hierárquica

Primeiro Princípio

O primeiro princípio proposto pelo IEEE (2004) foi o da organização hierárquica.

Estrutura Hierárquica

Estabelece que os componentes de uma proposta de solução ou a organização de elementos de um problema devem ser apresentados em formato hierárquico, em uma estrutura do tipo árvore, com quantidade crescente de detalhamento nível após nível.

Formalidade e Completeza

Formalidade

A formalidade estabelece que a resolução de um problema deve seguir uma abordagem rigorosa, metódica e padronizada.

Completeza

Completeza estabelece a necessidade de verificar cuidadosamente se todos os elementos de um problema foram levantados e se a solução proposta os contempla por completo.

Dividir para Conquistar

Princípio

O princípio de dividir para conquistar é fundamental na Engenharia de Software.

Justificativa

A justificativa é simples: fica intelectualmente inviável encarar um problema complexo sem que ele seja dividido em partes menores, independentes e, portanto, mais facilmente gerenciáveis.

Ocultação e Localização

Ocultação

Ao conceber uma solução modular, cada módulo deve ter acesso apenas às informações necessárias para seu funcionamento.

Localização

Localização aponta para a necessidade de se colocar juntos os itens relacionados logicamente em um sistema.

Integridade Conceitual

Princípio

O princípio da integridade conceitual estabelece que o engenheiro de software deve seguir uma filosofia e uma arquitetura de projeto coerentes.

IEEE

Conforme IEEE (2004), esse princípio é essencial para a coesão do projeto.

Abstração

Definição

Abstração determina a atenção do engenheiro de software a elementos com afinidade a um problema particular, isolando-os de outros elementos relacionados a outros tipos de problemas.

Exemplo

Pense na abstração como a capacidade de concentrar-se em aspectos essenciais para a resolução de um problema específico.

Exemplificação

Caso Prático

Certa equipe de desenvolvimento construiu um processador de texto. Ao planejar o menu da ferramenta, a funcionalidade de classificação por ordem alfabética acabou ficando no menu "Layout de página".

Erro

Dois princípios foram ignorados: a localização correta dos elementos no menu e a integridade conceitual, ao não adotar uma metodologia consistente do início ao fim do desenvolvimento.

O Software

Definição

De acordo com Pressmann e Maxim (2016), software de computador é o produto que profissionais de software desenvolvem e ao qual dão suporte no longo prazo.

Conteúdos

Abarca programas executáveis em um computador, conteúdos e informações descritivas.

Natureza Única

O software é um produto e, ao mesmo tempo, o veículo usado para a distribuição desse produto.

Elementos do Software

Elementos do Software

Software consiste em:

- Instruções que fornecem características, funções e desempenho desejados.
- Estruturas de dados que possibilitam aos programas manipular informações adequadamente.
- Informação descritiva, descrevendo a operação e o uso do programa.

Elementos do Software

Complexidade

Embora a criação de software pareça simples, desenvolver produtos de software é uma atividade sistemática, organizada e baseada em métodos.

Crise do Software

História

Na década de 1960, alguns atores do processo de desenvolvimento cunharam a expressão "crise do software" para evidenciar o momento adverso enfrentado na época.

Problemas

Programas que não funcionavam corretamente, processos falhos e dificuldades de manutenção eram comuns.

Requisitos

A qualidade do levantamento dos requisitos era baixa, acarretando incorreções no produto final.

Seleção Natural das Metodologias

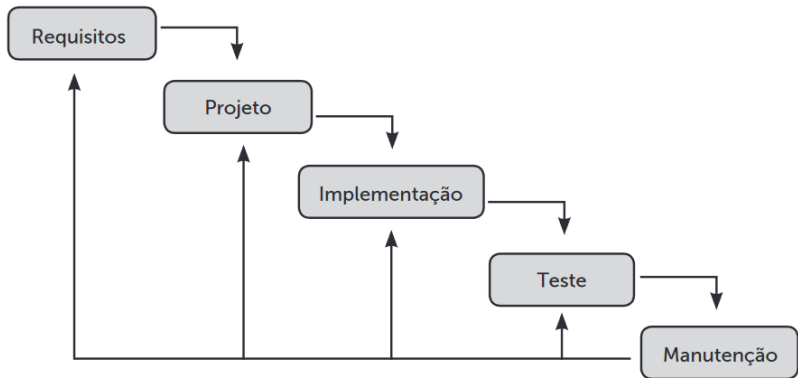
Evolução das Metodologias

Aquilo que não é bom não dura por muito tempo. Se a seleção natural é infalível na natureza, ela parece atuar com rigor semelhante no mundo do desenvolvimento de software ao permitir que apenas as metodologias mais aptas sobrevivam ao teste do tempo.

Modelo em Cascata

O Modelo em Cascata atravessou algumas décadas oferecendo aos engenheiros de software um modo razoavelmente seguro e efetivo de criar seus produtos, em substituição ao desenvolvimento especulativo e caótico praticado antes.

MODELO CASCATA



Limitações do Modelo em Cascata

Mudança de Paradigma

Nesta seção, entenderemos alguns elementos do Modelo em Cascata que justificaram a mudança de paradigma iniciada em 2001.

Falhas de Concepção

O Ciclo de Vida Tradicional, ou Modelo em Cascata, apresenta falhas de concepção que o tornaram obsoleto diante das demandas atuais de tempo e agilidade na produção de software.

Problemas da Metodologia Tradicional

Questões a Considerar

- Quais são os problemas da metodologia tradicional?
- Qual a dificuldade dela em acompanhar mudanças de requisitos?
- Por que o cliente tem dificuldade em reconhecer valor no que está sendo desenvolvido?

Objetivo

Mais do que responder a essas questões, esta seção aponta soluções e faz comparações adequadas.

Processo Tradicional de Desenvolvimento

Construção Linear

O processo tradicional de desenvolvimento baseia-se na construção linear do sistema, seguindo uma sequência definida de fases.

Etapas Sequenciais

Uma etapa utiliza o resultado obtido pela etapa anterior para criar seu artefato, podendo haver retornos para ajustes.

Linha de Montagem

A ideia é semelhante à de uma linha de montagem: a matéria-prima é inserida na linha de produção e, após etapas determinadas, o produto final está pronto.

ARTIGO

No Silver Bullet

O artigo "No Silver Bullet: Essence and Accidents of Software Engineering" de Frederick P. Brooks aborda a complexidade inerente ao desenvolvimento de software e a busca por soluções milagrosas.

Contexto

Publicação: 1986, época em que os métodos tradicionais dominavam o desenvolvimento de software.

Essência e Acidentes

Conceitos

- **Essência:** dificuldades inerentes ao problema de desenvolvimento de software.
- **Acidentes:** dificuldades derivadas do processo de implementação.

Importância

Entender a diferença entre essência e acidentes ajuda a identificar limitações dos métodos tradicionais.

Métodos Tradicionais

Características

- Modelos Lineares (Cascata)
- Documentação Extensiva
- Rigidez no Processo

Desvantagens

Dificuldade em lidar com mudanças de requisitos e alta formalidade.

Limitações dos Métodos Tradicionais

Problemas

- Ineficiência diante de mudanças rápidas
- Dificuldade na comunicação com o cliente
- Longo tempo de desenvolvimento

Exemplo

O Modelo em Cascata tem dificuldades em adaptar-se a novas necessidades do projeto durante o seu ciclo de vida.

Propostas de Brooks

Estratégias Sugeridas

- Prototipagem
- Desenvolvimento Incremental
- Ferramentas de Alto Nível

Objetivo

Reduzir acidentes e melhorar a produtividade sem buscar uma solução milagrosa.

Metodologias Ágeis

Características

- Desenvolvimento Iterativo
- Flexibilidade e Adaptação
- Foco na Colaboração e Comunicação

Exemplos

Extreme Programming (XP) e Scrum.

Comparação: Tradicional vs. Ágil

Métodos Tradicionais

- Estrutura Linear
- Documentação Extensiva
- Planejamento Detalhado

Abordagens Ágeis

- Iterações Curta
- Documentação Leve
- Adaptação Contínua

Vantagens das Metodologias Ágeis

Benefícios

- Melhor resposta a mudanças
- Aumento da colaboração
- Feedback contínuo

Resultado

Maior satisfação do cliente e entregas mais rápidas.

Exemplo de Transformação

Caso Prático

Uma empresa que adotou métodos ágeis passou a entregar software com maior frequência e qualidade, atendendo melhor às necessidades dos clientes.

Impacto

Redução no tempo de desenvolvimento e aumento da satisfação do cliente.

Conclusão

Reflexão

O artigo "No Silver Bullet" destaca que não existe uma solução única e milagrosa para os desafios do desenvolvimento de software.

Evolução

A transição de métodos tradicionais para abordagens ágeis mostra uma evolução na busca por eficiência e adaptação contínua.

Pergunta

Como sua organização pode se beneficiar das metodologias ágeis?